OOP- I. feladatsor

Tananyagtartalom

- OOP-fogalmak
  - Osztály, objektum, példányosítás.
  - Ősosztály, leszármazott osztály, öröklődés.
  - Konstruktor, az ősosztály konstruktorának meghívása: base(...).
  - Mező, háttérmező (backing field), tulajdonság.
  - get, set, csak olvasható tulajdonság.
  - Hozzáférési módosítók: public, private, protected.
  - Egységbezárás: belső állapot védelme, ellenőrzött módosítás.
  - Metódus, paraméter, visszatérési érték, void.
  - Virtuális metódus: virtual; felülírás: override.
  - Ősosztály metódusának hívása: base.Metodus().
  - Polimorfizmus: különböző számlatípusok használata Bankszamla típuson keresztül.
  - ToString() felülírása; GetType().Name.
- C#-os nyelvi elemek
  - decimal, string, bool, int; a decimal literál m jelölése.
  - Felsorolástípus: enum.
  - Statikus osztály és metódus: static; belépési pont: Main().
  - Névterek, using direktívák.
  - if, switch, switch kifejezés; for, foreach, while.
  - return, break.
  - out paraméter; TryParse().
  - Nullérték kezelése: ?, ?., ??, is null.
  - Szöveginterpoláció ($, {}); számformázás: N2.
  - Rövid tulajdonságmegadás: =>.
  - nameof().
- Hibakezelés és ellenőrzés
  - Bemenetellenőrzés, érvényes értéktartomány.
  - Kivételdobás: throw.
  - Kivételkezelés: try–catch.
  - ArgumentException, ArgumentOutOfRangeException, OverflowException.
  - Műveleti eredmény jelzése enum értékkel.
  - Határesetek; sikertelen műveletnél változatlan állapot.
- Adatkezelés és konzolos működés
  - Generikus lista: List<Bankszamla>.
  - Kollekcióinicializáló, indexelés, Count.
  - LINQ: Where, OrderByDescending, Sum, FirstOrDefault.
  - Kerekítés: Math.Round, MidpointRounding.AwayFromZero.
  - Magyar számformátum: CultureInfo; UTF-8 kimenet.
  - Konzolos adatbekérés, kiírás, menü.
- Tiszta kód alkalmazása
  - Beszédes elnevezések, rövid metódusok.
  - Ismétlődések csökkentése, közös működés újrafelhasználása.
  - Számlaműveletek és konzolos megjelenítés elkülönítése.

Feladatleírás

1. Hozd létre a `PenzfelvetelEredmeny` felsorolást (enum)!

Lehetséges értékei:

- `Sikeres`
- `ErvenytelenOsszeg`
- `LimitTullepve`
- `NincsElegFedezet`

2. Készítsd el a `Bankszamla` osztályt!

Tag	Típus	Elvárt működés
egyenleg	decimal	Privát mező
Tulajdonos	string	Nyilvánosan olvasható; csak a konstruktorban állítható
Egyenleg	decimal	Nyilvánosan olvasható; az egyenleg értékét adja
FelveteliLimit	decimal	Nyilvánosan olvasható és beállítható; csak pozitív lehet

A limithez használj privát háttérmezőt!

3. Készíts konstruktort!

Paraméterei: tulajdonos, kezdőegyenleg, felvételi limit.

Ellenőrzések:

- A tulajdonos neve nem lehet üres vagy csak szóközökből álló.
- A kezdőegyenleg nem lehet negatív.
- A limitnek pozitívnak kell lennie.

Érvénytelen argumentumnál dobj megfelelő kivételt: `ArgumentException` vagy `ArgumentOutOfRangeException`!

4. Készíts védett `Jovair(decimal osszeg)` segédmetódust!

- Csak pozitív összeget fogadjon el.
- Érvénytelen összegnél dobjon `ArgumentOutOfRangeException` kivételt.
- Növelje az egyenleget.
- Az osztály és a leszármazottai használhassák; a főprogram ne érhesse el.

5. Készíts nyilvános, virtuális `Befizet(decimal osszeg)` metódust!

- A befizetést a `Jovair()` segítségével végezze.
- Visszatérési típusa `void`.
- Ne írjon a konzolra.

6. Készíts védett `MegprobalTerhelni(decimal osszeg)` segédmetódust!

Visszatérési típusa `PenzfelvetelEredmeny`.

- Nem pozitív összeg: `ErvenytelenOsszeg`.
- Elégtelen egyenleg: `NincsElegFedezet`.
- Egyébként vonja le az összeget, majd adjon vissza `Sikeres` eredményt.
- Sikertelen műveletnél az egyenleg ne változzon.
- A felvételi limitet ez a segédmetódus ne vizsgálja: a teljes terhelést kezeli.

7. Készíts nyilvános, virtuális `Felvesz(decimal osszeg)` metódust!

Visszatérési típusa `PenzfelvetelEredmeny`.

Az ellenőrzések sorrendje:

1. Nem pozitív összeg: `ErvenytelenOsszeg`.
2. Felvételi limitet meghaladó összeg: `LimitTullepve`.
3. Egyébként a terhelést a `MegprobalTerhelni()` végezze.

8. Írd felül a `ToString()` metódust!

Jelenjen meg a számla típusa, tulajdonosa, egyenlege és felvételi limitje. A pénzösszegeket két tizedesjeggyel, `Ft` jelöléssel jelenítsd meg!

9. Próbáld ki az alaposztályt!

Hozz létre egy számlát:

- Tulajdonos: Kiss Anna
- Kezdőegyenleg: 50 000 Ft
- Felvételi limit: 30 000 Ft

Végezd el sorrendben:

Művelet	Elvárt eredmény	Egyenleg utána
10 000 Ft befizetése	Sikeres befizetés	60 000 Ft
20 000 Ft felvétele	`Sikeres`	40 000 Ft
35 000 Ft felvétele	`LimitTullepve`	40 000 Ft
0 Ft felvétele	`ErvenytelenOsszeg`	40 000 Ft
Limit módosítása 100 000 Ft-ra	Sikeres módosítás	40 000 Ft
50 000 Ft felvétele	`NincsElegFedezet`	40 000 Ft

A kivételek kipróbálásához a főprogramban kapj el egy negatív befizetésből és egy érvénytelen limitbeállításból származó kivételt. Írd ki az üzenetüket, majd folytasd a programot!

10. Készíts `Diakszamla` osztályt!

- Származzon a `Bankszamla` osztályból.
- Új tulajdonsága: `Iskola`, kívülről csak olvasható.

- Konstruktorparaméterei: tulajdonos, kezdőegyenleg, iskola.
- Felvételi limitje automatikusan 20 000 Ft legyen.
- Az iskola neve nem lehet üres vagy csak szóközökből álló.
- Használd az ősosztály konstruktorát!
- A `ToString()` egészítse ki az alapadatokat az iskola nevével.

11. Készíts `PremiumSzamla` osztályt!

- Származzon a `Bankszamla` osztályból.
- Új tulajdonsága: `BonuszSzazalek`, kívülről csak olvasható.
- Konstruktorparaméterei: tulajdonos, kezdőegyenleg, limit, bónuszszázalék.
- A bónuszszázalék 0 és 10 közötti érték lehet, a határokat is beleértve.
- Írd felül a `Befizet()` metódust!
- Az eredeti összeget az ősosztály metódusával fizesd be.
- A bónuszt a védett `Jovair()` segítségével írd jóvá, ha pozitív.
- A bónuszt két tizedesjegyre, `MidpointRounding.AwayFromZero` szabállyal kerekítsd.
- A kezdőegyenlegre ne járjon bónusz.
- A `ToString()` jelenítse meg a bónuszszázalékot is.

12. Készíts `UzletiSzamla` osztályt!

- Származzon a `Bankszamla` osztályból.
- Új, kívülről csak olvasható tulajdonságai:
  - `Cegnev`
  - `PenzfelveteliDij`
- Konstruktorparaméterei: tulajdonos, kezdőegyenleg, limit, cégnév, pénzfelvételi díj.
- A cégnév nem lehet üres vagy csak szóközökből álló.
- A díj nem lehet negatív.
- Írd felül a `Felvesz()` metódust!
- Ellenőrizd a kért összeg pozitivitását és a limitet.
- A limit **a felvenni kívánt összegre** vonatkozzon, a díj nélkül.
- Az összeg és a díj együttes levonását a `MegprobalTerhelni()` végezze.
- Ha nincs elegendő fedezet a teljes terhelésre, semmit se vonj le.
- A `ToString()` jelenítse meg a cégnevet és a díjat is.

13. Ellenőrizd a speciális működéseket!

Számla és művelet	Elvárt eredmény
Prémiumszámla: 10 000 Ft egyenleg, 2% bónusz; 5 000 Ft befizetése	Új egyenleg: 15 100 Ft
Diákszámla: 30 000 Ft egyenleg; 25 000 Ft felvétele	`LimitTullepve`; változatlan egyenleg
Üzleti számla: 10 000 Ft egyenleg, 500 Ft díj, 20 000 Ft limit; 9 500 Ft felvétele	`Sikeres`; új egyenleg: 0 Ft
Ugyanilyen, új üzleti számla; 10 000 Ft felvétele	`NincsElegFedezet`; egyenleg: 10 000 Ft

14. Hozz létre egy közös `List<Bankszamla>` listát!

Legalább négy objektumot tartalmazzon:

- hagyományos bankszámla;
- diákszámla;
- prémiumszámla;
- üzleti számla.

15. Jelenítsd meg a számlákat!

- `foreach` ciklussal járd be a listát.
- Használd a felülírt `ToString()` metódusokat.
- A kiíráshoz ne készíts számlatípusonként külön elágazásokat.

16. Végezz közös műveleteket!

- Minden számlára fizess be 10 000 Ft-ot.
- Minden számláról próbálj felvenni 15 000 Ft-ot.
- A hívások `Bankszamla` típusú változón keresztül történjenek.
- Figyeld meg a bónusz és a pénzfelvételi díj hatását!

17. Készíts eredményüzenetet adó metódust a főprogramban!

- Paramétere: `PenzfelvetelEredmeny`.
- Visszatérési értéke: érthető magyar üzenet.
- A konzolra írás a főprogram feladata maradjon.
- Minden lehetséges eredményt kezelj.

18. Készíts LINQ-lekérdezéseket!

- Számlák egyenlegének összege.
- Legnagyobb egyenlegű számla.
- 30 000 Ft feletti egyenlegű számlák, egyenleg szerint csökkenő sorrendben.

19. Ellenőrizd és javítsd a kódot!

- Sikertelen pénzfelvételnél változatlan egyenleg.
- Pontosan elegendő fedezet esetén sikeres művelet.
- Limittel azonos összeg felvehető.
- Nulla százalékos bónusz és nulla forintos díj helyes kezelése.
- Beszédes nevek, rövid metódusok, közös működés újrafelhasználása.
- A számlaosztályokban nincs `Console.WriteLine()` vagy adatbekérés.

Szorgalmi

Egyszerű konzolos menü; számla kiválasztása, befizetés, pénzfelvétel, adatok megjelenítése.